

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART V KUBERNETES RESOURCES

Every Core Resource: Deep Dive with YAML, Security, and Best Practices

Volume 5 of 16 Core Series

Prerequisites: Parts I through IV

Edition 2025-2026

TABLE OF CONTENTS

1. Resource Taxonomy and API Conventions	3
2. Pods -- The Atomic Unit	5
3. Workload Resources: Deployment, StatefulSet, DaemonSet .	12
4. Batch Resources: Jobs and CronJobs	20
5. Service and Endpoint Resources	24
6. Ingress and Gateway API	29
7. Configuration: ConfigMaps and Secrets	34
8. Storage: PV, PVC, StorageClass, CSI	38
9. Namespaces, ResourceQuota, LimitRange	44
10. Identity and Access: ServiceAccounts and RBAC ..	48
11. Network Policies	53
12. Autoscaling: HPA, VPA, KEDA, Cluster Autoscaler .	57
13. Scheduling: PriorityClass, Taints, Affinity, TopologySpread .	63
14. Reliability: PodDisruptionBudgets	67
15. Runtime and Policy: RuntimeClass, PodSecurity ..	70
16. Resource Anti-Patterns Reference	73
17. Hands-On Exercises	76

CHAPTER 1

Resource Taxonomy and API Conventions

Every Kubernetes resource follows a consistent structure and set of conventions. Mastering these conventions allows you to work with any resource -- built-in or custom -- without memorising each one individually.

Universal Resource Structure

```
# Every Kubernetes resource has this structure: apiVersion: GROUP/VERSION # e.g. apps/v1, v1, batch/v1
kind: RESOURCE_KIND # e.g. Deployment, Pod, Service
metadata: name: my-resource # unique within namespace
namespace: default # omit for cluster-scoped resources
labels: # key-value pairs for selection
app: myapp
version: v1
annotations: # non-identifying metadata
deployment.kubernetes.io/revision: '3'
ownerReferences: # garbage collection parent -
apiVersion: apps/v1
kind: Deployment
name: myapp
uid: abc123
controller: true
finalizers: # prevent deletion until cleanup -
kubernetes.io/pvc-protection
resourceVersion: '12345' # optimistic concurrency token
uid: abc-def-123 # immutable unique identifier
spec: # DESIRED state (user-controlled) ...
status: # ACTUAL state (system-controlled) ...
```

Resource Scope and Taxonomy

Category	Resources	Scope	Primary Purpose
Workload	Pod, Deployment, StatefulSet, DaemonSet, ReplicaSet	Namespaced	Run containerised workloads
Batch	Job, CronJob	Namespaced	Run finite or scheduled tasks
Networking	Service, Endpoints, EndpointSlice, Ingress, NetworkPolicy	Namespaced	Expose and control traffic
Gateway	Gateway, HTTPRoute, GRPCRoute, TCPRoute	Namespaced	Next-gen ingress/routing
Config	ConfigMap, Secret	Namespaced	Inject configuration and secrets
Storage	PersistentVolume, StorageClass	Cluster	Persistent storage abstraction
Storage	PersistentVolumeClaim	Namespaced	Claim storage for a workload
Identity	ServiceAccount, Secret (token)	Namespaced	Workload identity and auth
RBAC	Role, RoleBinding	Namespaced	Namespace-scoped access control
RBAC	ClusterRole, ClusterRoleBinding	Cluster	Cluster-wide access control

Category	Resources	Scope	Primary Purpose
Policy	LimitRange, ResourceQuota	Namespaced	Resource governance per namespace
Policy	PodDisruptionBudget	Namespaced	Availability during disruptions
Autoscaling	HPA, VPA	Namespaced	Automatic resource scaling
Scheduling	PriorityClass	Cluster	Pod preemption priority
Runtime	RuntimeClass	Cluster	Container runtime selection
Nodes	Node, CSINode	Cluster	Worker node representation
Cluster	Namespace, PersistentVolume	Cluster	Cluster-wide organisation

CHAPTER 2

Pods -- The Atomic Unit

A Pod is the smallest deployable unit in Kubernetes. It represents one or more containers that share a network namespace (same IP address and port space), IPC namespace, and optionally a PID namespace. Containers in a Pod are always co-located, co-scheduled, and share storage volumes.

Complete Production Pod Specification

```
apiVersion: v1 kind: Pod metadata: name: production-app namespace: production labels: app: myapp version: v1.2.3 tier: backend spec: # Security at Pod level securityContext: runAsNonRoot: true runAsUser: 1000 runAsGroup: 1000 fsGroup: 2000 seccompProfile: type: RuntimeDefault # Scheduling controls serviceAccountName: myapp-sa automountServiceAccountToken: false # Explicit -- don't auto-mount terminationGracePeriodSeconds: 60 nodeSelector: kubernetes.io/arch: amd64 tolerations: - key: node.kubernetes.io/not-ready operator: Exists effect: NoExecute tolerationSeconds: 300 # Init containers (run before app containers, in order) initContainers: - name: wait-for-db image: busybox:1.36 command: ['sh', '-c', 'until nc -z db-service 5432; do sleep 2; done'] securityContext: allowPrivilegeEscalation: false capabilities: { drop: [ALL] } # App containers containers: - name: app image: harbor.corp/myapp@sha256:abc123 ports: - name: http containerPort: 8080 protocol: TCP env: - name: DB_HOST value: db-service.production.svc.cluster.local - name: DB_PASSWORD valueFrom: secretKeyRef: name: db-credentials key: password envFrom: - configMapRef: name: app-config resources: requests: cpu: 250m memory: 256Mi limits: memory: 512Mi # CPU limit often omitted to avoid throttling # Health probes startupProbe: httpGet: { path: /health, port: 8080 } failureThreshold: 30 periodSeconds: 10 # 30*10s = 5min max startup time livenessProbe: httpGet: { path: /health, port: 8080 } initialDelaySeconds: 0 periodSeconds: 15 failureThreshold: 3 readinessProbe: httpGet: { path: /ready, port: 8080 } periodSeconds: 5 failureThreshold: 3 # Lifecycle hooks lifecycle: preStop: exec: command: ['/bin/sh', '-c', 'sleep 5'] # Allow LB drain # Security at container level securityContext: allowPrivilegeEscalation: false readOnlyRootFilesystem: true capabilities: drop: [ALL] volumeMounts: - name: tmp mountPath: /tmp - name: cache mountPath: /app/cache - name: config mountPath: /app/config readOnly: true volumes: - name: tmp emptyDir: { medium: Memory } - name: cache emptyDir: {} - name: config configMap: name: app-config
```

Pod Health Probe Decision Guide

Probe	Triggers On Failure	When to Use	Recommended Check
startupProbe	Nothing (waits for success)	Slow-starting apps (JVM, large models)	Same endpoint as liveness; high failureThreshold
livenessProbe	Container restart	Detect deadlocks, hung processes	Lightweight internal health check; never external deps
readinessProbe	Remove from Service endpoints	Temporary unreadiness (DB reconnect, warmup)	Include dependency checks (DB, cache reachable)

Multi-Container Pod Patterns

- * **Sidecar:** A helper container augmenting the main container: log shipper, metrics exporter, secret refresher, Istio Envoy proxy. Shares Pod lifetime.
- * **Init Container:** Runs before app containers to perform setup: wait for dependencies, populate a shared volume, run database migrations. Runs once to completion.
- * **Sidecar (native, K8s 1.29+):** New native sidecar type: init container with restartPolicy: Always. Starts before app containers, stops after them. Solves ordering issue with Istio Envoy and log shippers.
- * **Ephemeral Container:** Debug container added to running Pod without restart. kubectl debug in-cluster. Does not persist; cannot use volumes.
- * **Ambassador:** Proxy container that simplifies external service access. App connects to localhost; ambassador handles service discovery and auth.

CHAPTER 3

Workload Resources: Deployment, StatefulSet, DaemonSet

Deployment -- Stateless Workloads

Deployments manage stateless application replicas. They provide rolling updates, rollback capability, and scaling. Under the hood, a Deployment manages ReplicaSets which manage Pods.

```
apiVersion: apps/v1 kind: Deployment metadata: name: api-server namespace: production spec:
replicas: 3 revisionHistoryLimit: 5 # Keep 5 old ReplicaSets for rollback
progressDeadlineSeconds: 600 # Fail rollout if not complete in 10min selector: matchLabels:
app: api-server strategy: type: RollingUpdate rollingUpdate: maxSurge: 1 # +1 Pod above
replicas during update maxUnavailable: 0 # Zero-downtime: no Pods removed until new ready
template: metadata: labels: app: api-server version: v1.2.3 spec: # Anti-affinity: spread Pods
across nodes affinity: podAntiAffinity: preferredDuringSchedulingIgnoredDuringExecution: -
weight: 100 podAffinityTerm: labelSelector: matchLabels: { app: api-server } topologyKey:
kubernetes.io/hostname containers: - name: api image: harbor.corp/api@sha256:abc resources:
requests: { cpu: 500m, memory: 512Mi } limits: { memory: 1Gi }
```

StatefulSet -- Stateful Workloads

StatefulSets manage stateful applications that require stable network identities, stable persistent storage, and ordered deployment/scaling. The canonical use cases are databases, message queues, and distributed caches.

StatefulSet Guarantees vs. Deployment

Property	Deployment	StatefulSet
Pod naming	Random suffix (api-7d9f8-abc12)	Ordinal index (postgres-0, postgres-1)
DNS	Shared Service ClusterIP	Per-Pod DNS: pod-0.headless-svc.ns.svc.cluster.local
Storage	Shared or no PVC per pod	Dedicated PVC per Pod (volumeClaimTemplates)
Startup order	Parallel (all at once)	Sequential (0 then 1 then 2)
Shutdown order	Random/parallel	Reverse ordinal (2 then 1 then 0)
Pod identity	Ephemeral (any Pod is equivalent)	Sticky (postgres-0 always postgres-0)
Rolling update	All Pods updated with surge	Reverse ordinal (highest first)
Use cases	Web servers, APIs, stateless workers	Databases, Kafka, Elasticsearch, etcd

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: postgres spec: serviceName:
postgres-headless # REQUIRED: headless Service for DNS replicas: 3 selector: matchLabels: {
app: postgres } updateStrategy: type: RollingUpdate rollingUpdate: partition: 0 # Update all
```



```

pods (set to N to protect first N pods) template: metadata: labels: { app: postgres } spec:
containers: - name: postgres image: postgres:16-alpine env: - name: POD_NAME valueFrom: {
fieldRef: { fieldPath: metadata.name } } - name: POSTGRES_PASSWORD valueFrom: { secretKeyRef:
{ name: pg-secret, key: password } } volumeMounts: - name: data mountPath:
/var/lib/postgresql/data resources: requests: { cpu: 1, memory: 2Gi } limits: { memory: 4Gi }
# Dedicated PVC per Pod -- created automatically, never deleted on scale-down
volumeClaimTemplates: - metadata: name: data spec: accessModes: [ReadWriteOnce]
storageClassName: fast-ssd resources: requests: storage: 100Gi

```

DaemonSet -- Node-Level Workloads

DaemonSets ensure exactly one Pod runs on every node (or every node matching a selector). DaemonSets are used for node-level infrastructure: log collection, monitoring agents, network plugins, storage drivers, security agents.

Use Case	Example DaemonSet	Notes
Log collection	Fluent Bit, Fluentd	One per node to collect /var/log/pods/
Metrics collection	node_exporter, DCGM GPU exporter	Node hardware/OS metrics
Networking	Cilium, Calico, Flannel agents	CNI plugin node agent
Storage	CSI node driver (EBS, Ceph RBD)	Volume attach/mount on node
Security	Falco, Tetragon	Kernel-level eBPF probes
GPU management	NVIDIA device plugin	Expose GPU resources to scheduler
Cluster DNS	CoreDNS (can also run as Deployment)	Per-node DNS caching

CHAPTER 4

Batch Resources: Jobs and CronJobs

Jobs -- Run-to-Completion Workloads

A Job creates one or more Pods that run to completion. Unlike Deployments (which keep Pods running), Jobs are designed for finite tasks: database migrations, batch data processing, ML training runs, report generation.

```
apiVersion: batch/v1 kind: Job metadata: name: data-migration-v2 spec: completions: 1 # Total
successful completions required parallelism: 1 # Pods running in parallel backoffLimit: 3 #
Retry failed pods up to 3 times activeDeadlineSeconds: 3600 # Kill job if running > 1 hour
ttlSecondsAfterFinished: 86400 # Auto-delete 24h after completion template: spec:
restartPolicy: OnFailure # Never or OnFailure (not Always) containers: - name: migrator image:
harbor.corp/migrator:v2 command: ['python', 'migrate.py', '--version', 'v2'] resources:
requests: { cpu: 500m, memory: 1Gi } # Parallel Job patterns: # Fixed completion count:
completions=10, parallelism=3 (process 10 tasks, 3 at a time) # Work queue: completions=1,
parallelism=5 (5 workers drain a queue) # Indexed Jobs (1.21+): completionMode: Indexed (each
Pod gets JOB_COMPLETION_INDEX)
```

AI/ML Training Job Pattern

```
# Distributed PyTorch training Job: apiVersion: batch/v1 kind: Job metadata: name:
llm-finetune-run-001 spec: completions: 8 # 8 workers total parallelism: 8 # All start
simultaneously (gang scheduling) completionMode: Indexed template: spec: schedulerName:
volcano # Gang scheduler restartPolicy: OnFailure tolerations: - key: nvidia.com/gpu operator:
Exists effect: NoSchedule containers: - name: trainer image: nvcr.io/nvidia/pytorch:24.05-py3
command: - torchrun --nproc_per_node=8 --nnodes=1 - train.py resources: limits:
nvidia.com/gpu: 8 requests: nvidia.com/gpu: 8 memory: 256Gi cpu: 64
```

CronJobs

CronJobs create Jobs on a schedule (cron syntax). Used for periodic tasks: database backups, report generation, cache warming, cleanup jobs.

```
apiVersion: batch/v1 kind: CronJob metadata: name: database-backup spec: schedule: '0 2 * * *'
# 02:00 UTC daily timeZone: 'UTC' # Explicit timezone (K8s 1.27+) concurrencyPolicy: Forbid #
Skip if previous run still active successfulJobsHistoryLimit: 3 failedJobsHistoryLimit: 1
startingDeadlineSeconds: 300 # Skip if > 5min late to start jobTemplate: spec:
ttlSecondsAfterFinished: 3600 template: spec: restartPolicy: OnFailure containers: - name:
backup image: harbor.corp/pg-backup:latest env: - name: BACKUP_DESTINATION value:
s3://backups/postgres/ resources: requests: { cpu: 200m, memory: 256Mi }
```

CHAPTER 5

Service and Endpoint Resources

Services provide stable network identities (IP addresses and DNS names) for groups of Pods. Without Services, Pods would need to track each other's ephemeral IPs directly -- an impossible task at scale.

Service Types Reference

```
# ClusterIP (default) -- internal cluster communication: apiVersion: v1 kind: Service
metadata: name: api-backend spec: type: ClusterIP selector: app: api-backend version: v1
ports: - name: http port: 80 # Port exposed by Service targetPort: 8080 # Port on Pod
protocol: TCP sessionAffinity: None # Or ClientIP for sticky sessions # NodePort -- expose on
each node's IP: spec: type: NodePort ports: - port: 80 targetPort: 8080 nodePort: 30080 # Must
be 30000-32767; omit for auto-assign # LoadBalancer -- provision cloud LB: spec: type:
LoadBalancer loadBalancerSourceRanges: - 10.0.0.0/8 # Restrict LB to internal networks
externalTrafficPolicy: Local # Preserve client source IP # Headless Service -- direct Pod DNS
(for StatefulSets): spec: clusterIP: None # No virtual IP; DNS returns Pod IPs directly
selector: app: postgres # ExternalName -- DNS CNAME to external service: spec: type:
ExternalName externalName: legacy-api.internal.corp
```

EndpointSlices -- How Services Find Pods

EndpointSlices (replaced Endpoints in 1.21+) track the IP addresses and ports of Pods matching a Service's selector. kube-proxy watches EndpointSlices to update its routing rules. The EndpointSlice controller creates/updates them as Pods come and go:

```
# View EndpointSlices for a Service: kubectl get endpointslices -l
kubernetes.io/service-name=api-backend # Each EndpointSlice holds up to 100 endpoints (for
scalability): apiVersion: discovery.k8s.io/v1 kind: EndpointSlice metadata: name:
api-backend-abc labels: kubernetes.io/service-name: api-backend addressType: IPv4 endpoints: -
addresses: ['10.244.1.5'] conditions: ready: true serving: true terminating: false targetRef:
{ kind: Pod, name: api-backend-xyz, namespace: default } hints: forZones: [{ name: us-east-1a
}] # Topology hints for zone-aware routing ports: - name: http port: 8080 protocol: TCP
```

CHAPTER 6

Ingress and Gateway API

Ingress and Gateway API provide HTTP-level routing from external traffic into cluster Services. Ingress is the original API (stable since 1.19); Gateway API is the successor with richer routing semantics and role-based management.

Ingress

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: api-ingress annotations:
nginx.ingress.kubernetes.io/ssl-redirect: 'true' nginx.ingress.kubernetes.io/proxy-body-size:
100m nginx.ingress.kubernetes.io/rate-limit: '100' cert-manager.io/cluster-issuer:
letsencrypt-prod spec: ingressClassName: nginx tls: - hosts: [api.example.com] secretName:
api-tls-cert rules: - host: api.example.com http: paths: - path: /v1 pathType: Prefix backend:
service: name: api-v1 port: { number: 80 } - path: /v2 pathType: Prefix backend: service:
name: api-v2 port: { number: 80 }
```

Gateway API -- The Ingress Successor

Gateway API (stable in 1.28) separates infrastructure (Gateway) from routing (HTTPRoute) and supports multi-tenancy through role separation. Different teams manage different parts of the routing configuration:

```
# Infrastructure team manages the Gateway (allocates LB): apiVersion:
gateway.networking.k8s.io/v1 kind: Gateway metadata: name: external-gateway namespace:
ingress-system spec: gatewayClassName: cilium # or nginx, envoy, istio listeners: - name:
https protocol: HTTPS port: 443 tls: certificateRefs: - name: wildcard-tls namespace:
ingress-system allowedRoutes: namespaces: from: Selector selector: matchLabels: {
gateway-allowed: 'true' } # Application team manages their own HTTPRoute: apiVersion:
gateway.networking.k8s.io/v1 kind: HTTPRoute metadata: name: api-routes namespace: production
spec: parentRefs: - name: external-gateway namespace: ingress-system hostnames:
['api.example.com'] rules: - matches: - path: { type: PathPrefix, value: /api } headers: -
name: X-API-Version value: v2 backendRefs: - name: api-v2 port: 80 weight: 90 - name:
api-v3-canary port: 80 weight: 10 filters: - type: RequestHeaderModifier
requestHeaderModifier: add: - name: X-Forwarded-By value: gateway
```

CHAPTER 7

Configuration: ConfigMaps and Secrets

ConfigMaps -- Externalised Non-Secret Configuration

```
apiVersion: v1 kind: ConfigMap metadata: name: app-config namespace: production data: # Simple
key-value pairs (used as env vars): LOG_LEVEL: info MAX_CONNECTIONS: '100' FEATURE_FLAGS:
'new-ui=true,beta-api=false' # File content (mounted as file in container): nginx.conf: |
server { listen 8080; location / { proxy_pass http://backend; } } app.properties: |
database.url=jdbc:postgresql://db:5432/mydb cache.ttl=300 # Consume as env vars: envFrom: -
configMapRef: { name: app-config } # Or selectively: env: - name: LOG_LEVEL valueFrom:
configMapKeyRef: { name: app-config, key: LOG_LEVEL } # Mount as files: volumes: - name:
config configMap: name: app-config items: - key: nginx.conf path: nginx.conf mode: 0444
```

Secrets -- Sensitive Configuration

Kubernetes Secrets store sensitive data (passwords, TLS certs, API keys). Secrets are base64-encoded (NOT encrypted) by default -- encryption at rest requires additional configuration or an external secrets manager.

Secret Security Warning

Kubernetes Secrets are NOT encrypted by default -- only base64-encoded. Anyone with RBAC access to read Secrets can decode them. Enable etcd encryption at rest (EncryptionConfiguration) and use External Secrets Operator with HashiCorp Vault, AWS Secrets Manager, or GCP Secret Manager for production secret management. Never commit Secrets to Git.

```
# Secret types: # Opaque: generic key-value (most common) # kubernetes.io/tls: TLS certificate
+ private key # kubernetes.io/dockerconfigjson: registry pull secret #
kubernetes.io/service-account-token: SA token (auto-created) apiVersion: v1 kind: Secret
metadata: name: db-credentials namespace: production type: Opaque stringData: # kubectl
encodes to base64 automatically username: dbadmin password: my-secure-password
connection-string: postgresql://dbadmin:my-secure-password@db:5432/mydb # TLS Secret (for
Ingress/Gateway TLS): apiVersion: v1 kind: Secret metadata: name: api-tls type:
kubernetes.io/tls data: tls.crt: tls.key:
```

External Secrets Operator -- Production Secret Management

```
apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: db-credentials
namespace: production spec: refreshInterval: 1h secretStoreRef: name: vault-backend kind:
ClusterSecretStore target: name: db-credentials creationPolicy: Owner deletionPolicy: Delete
data: - secretKey: password remoteRef: key: secret/production/database property: password
```

CHAPTER 8

Storage: PV, PVC, StorageClass, CSI

Kubernetes abstracts storage through a three-layer model: StorageClass (defines storage capabilities), PersistentVolume (represents actual storage), and PersistentVolumeClaim (requests storage for a workload). CSI drivers implement the actual storage operations.

Storage Architecture

```
Dynamic provisioning flow: 1. StorageClass defined (once, by storage admin): parameters,
provisioner, reclaimPolicy, volumeBindingMode 2. PVC created (by developer): accessModes,
resources.requests.storage, storageClassName 3. PVC controller detects unbound PVC 4. CSI
provisioner creates volume in backend (AWS EBS, GCS, Ceph) 5. PV created automatically, bound
to PVC 6. Pod created referencing PVC 7. kubelet calls CSI NodePublishVolume on scheduled node
8. Volume attached to node (CSI ControllerPublishVolume) 9. Volume mounted into Pod filesystem
# StorageClass example (AWS EBS): apiVersion: storage.k8s.io/v1 kind: StorageClass metadata:
name: fast-ssd annotations: storageclass.kubernetes.io/is-default-class: 'true' provisioner:
ebs.csi.aws.com parameters: type: gp3 iops: '3000' throughput: '125' encrypted: 'true'
kmsKeyId: arn:aws:kms:us-east-1:ACCOUNT:key/KEY-ID reclaimPolicy: Delete # Delete PV when PVC
deleted (use Retain for prod) volumeBindingMode: WaitForFirstConsumer # Provision on node with
Pod allowVolumeExpansion: true
```

Access Modes

Mode	Abbreviation	Semantics	Typical Use
ReadWriteOnce	RWO	Read/write by one node at a time	Databases, single-instance apps
ReadOnlyMany	ROX	Read-only by many nodes	Shared config files, static assets
ReadWriteMany	RWX	Read/write by many nodes simultaneously	Shared ML datasets, NFS workloads
ReadWriteOncePod	RWOP	Read/write by exactly one Pod	Strongest guarantee; K8s 1.22+

AI/ML Storage Patterns

- * **Model artifacts:** Large read-only model weights (10-700GB). Store in object storage (S3, GCS). Mount read-only via CSI or init container download. Consider ReadOnlyMany PVC backed by NFS or parallel filesystem for multi-replica serving.
- * **Training datasets:** Massive datasets (TB scale). Use ReadOnlyMany PVCs backed by parallel filesystems (Lustre, GPFS, WekaFS) for high-throughput sequential read. Or stream from object storage (S3) with dataset caching layer.
- * **Checkpoints:** Write-heavy during training, read-heavy during restart. Use ReadWriteOnce SSD PVC for low-latency checkpoint writes. Async copy to object storage for durability.

- * **Vector databases:** High-performance read/write with specific IOPS requirements. Use gp3 EBS (AWS) or Premium SSD (Azure) with StorageClass IOPS tuning. Size for 2-3x index size for memory-mapped files.

CHAPTER 9

Namespaces, ResourceQuota, LimitRange

Namespaces provide a virtual cluster within a physical cluster. They partition resources between teams, environments, or tenants. ResourceQuota and LimitRange provide the governance layer: controlling how much resource a namespace can consume and enforcing default resource specifications.

Namespace Strategy

Pattern	Namespaces	Isolation	Use Case
Per environment	dev, staging, production	Low (same cluster)	Small teams, simple workloads
Per team	team-a, team-b, shared	Medium	Multi-team platform
Per application	myapp-dev, myapp-prod	Medium	Application-centric teams
Per tenant	tenant-acme, tenant-globex	Medium-High	SaaS multi-tenancy
Per cluster	(all in one namespace)	Highest	Dedicated cluster per env/tenant

ResourceQuota -- Namespace Resource Limits

```
apiVersion: v1 kind: ResourceQuota metadata: name: production-quota namespace: production spec: hard: # Compute: requests.cpu: '20' # Total CPU requests across all Pods limits.cpu: '40' requests.memory: 40Gi limits.memory: 80Gi # GPU (requires device plugin): requests.nvidia.com/gpu: '8' # Storage: requests.storage: 500Gi persistentvolumeclaims: '20' # Object counts: pods: '50' services: '20' services.loadbalancers: '2' configmaps: '30' secrets: '30' # Check quota usage: kubectl describe resourcequota -n production
```

LimitRange -- Per-Pod Defaults and Constraints

```
apiVersion: v1 kind: LimitRange metadata: name: default-limits namespace: production spec: limits: - type: Container # Defaults applied when not specified: defaultRequest: cpu: 100m memory: 128Mi default: cpu: 500m memory: 256Mi # Allowed ranges: min: cpu: 50m memory: 64Mi max: cpu: '4' memory: 8Gi - type: PersistentVolumeClaim min: { storage: 1Gi } max: { storage: 100Gi }
```


CHAPTER 10

Identity and Access: ServiceAccounts and RBAC

RBAC (Role-Based Access Control) is the primary authorisation mechanism in Kubernetes. It controls which identities (users, groups, service accounts) can perform which operations on which resources. ServiceAccounts provide identity for Pods.

RBAC Model

```
RBAC primitives: Role: namespaced permissions ClusterRole: cluster-wide permissions (or shared
across namespaces) RoleBinding: bind Role or ClusterRole to subjects (namespaced)
ClusterRoleBinding: bind ClusterRole to subjects (cluster-wide) Subject: User | Group |
ServiceAccount # Example: Allow a ServiceAccount to read Pods in its namespace apiVersion:
rbac.authorization.k8s.io/v1 kind: Role metadata: name: pod-reader namespace: production
rules: - apiGroups: [''] resources: [pods, pods/log] verbs: [get, list, watch] --- apiVersion:
rbac.authorization.k8s.io/v1 kind: RoleBinding metadata: name: pod-reader-binding namespace:
production subjects: - kind: ServiceAccount name: monitoring-agent namespace: production
roleRef: kind: Role name: pod-reader apiGroup: rbac.authorization.k8s.io
```

ServiceAccount Workload Identity

```
# Minimal ServiceAccount for an application: apiVersion: v1 kind: ServiceAccount metadata:
name: myapp-sa namespace: production annotations: # Workload Identity (GKE) -- bind to GCP
service account: iam.gke.io/gcp-service-account: myapp@project.iam.gserviceaccount.com # IRSA
(EKS) -- bind to AWS IAM role: eks.amazonaws.com/role-arn: arn:aws:iam::ACCOUNT:role/MyAppRole
automountServiceAccountToken: false # Opt-in, not opt-out # Pod using the ServiceAccount:
spec: serviceAccountName: myapp-sa automountServiceAccountToken: true # Explicit opt-in for
this pod # RBAC best practices: # 1. Principle of least privilege: grant minimum required
verbs # 2. Never use cluster-admin for application service accounts # 3. Separate
ServiceAccount per application # 4. Audit with: kubectl auth can-i
--as=system:serviceaccount:ns:sa # 5. Use Workload Identity (GKE) / IRSA (EKS) instead of
mounting cloud credentials
```

CHAPTER 11

Network Policies

NetworkPolicies are the Kubernetes firewall. They define which Pods can communicate with which other Pods and external endpoints. By default, all Pod-to-Pod communication is allowed (no NetworkPolicies = no restrictions). Applying a NetworkPolicy to a Pod makes that Pod's traffic subject to the policy's rules.

Default Deny -- Zero Trust Starting Point

```
# Apply to a namespace to block all traffic by default: # Then add explicit allow rules for
required communication apiVersion: networking.k8s.io/v1 kind: NetworkPolicy metadata: name:
default-deny-all namespace: production spec: podSelector: {} # Selects ALL pods in namespace
policyTypes: - Ingress - Egress # No ingress/egress rules = deny all
```

Microservices Network Policy Pattern

```
# Allow frontend to reach backend only: apiVersion: networking.k8s.io/v1 kind: NetworkPolicy
metadata: name: backend-ingress namespace: production spec: podSelector: matchLabels: { tier:
backend } policyTypes: [Ingress] ingress: - from: - podSelector: matchLabels: { tier: frontend
} - namespaceSelector: matchLabels: { team: monitoring } ports: - protocol: TCP port: 8080 #
Allow egress to DNS and external API only: apiVersion: networking.k8s.io/v1 kind:
NetworkPolicy metadata: name: backend-egress namespace: production spec: podSelector:
matchLabels: { tier: backend } policyTypes: [Egress] egress: - to: - namespaceSelector:
matchLabels: { kubernetes.io/metadata.name: kube-system } podSelector: matchLabels: { k8s-app:
kube-dns } ports: [{ port: 53, protocol: UDP }] - to: - ipBlock: cidr: 10.0.0.0/8 # Internal
services only ports: [{ port: 443, protocol: TCP }]
```

CHAPTER 12

Autoscaling: HPA, VPA, KEDA, Cluster Autoscaler

Autoscaling Layers

Scaler	Dimension	Trigger	Response Time
HPA (Horizontal Pod Autoscaler)	Pod count	CPU, memory, custom/external metrics	30-60 seconds
VPA (Vertical Pod Autoscaler)	Pod CPU/memory requests	Historical resource usage	Hours (requires restart)
KEDA	Pod count	Event sources (Kafka lag, SQS depth, Redis, Prometheus)	10-30 seconds
Cluster Autoscaler	Node count	Unschedulable pods; underutilised nodes	1-5 minutes
Karpenter	Node count + type	Unschedulable pods (intelligent bin-packing)	30-90 seconds

HPA -- Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: api-hpa namespace:
production spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: api-server
minReplicas: 3 maxReplicas: 50 metrics: - type: Resource resource: name: cpu target: type:
Utilization averageUtilization: 70 # Scale when avg CPU > 70% of request - type: Resource
resource: name: memory target: type: AverageValue averageValue: 400Mi - type: Pods pods:
metric: name: http_requests_per_second target: type: AverageValue averageValue: '1000'
behavior: scaleDown: stabilizationWindowSeconds: 300 # Wait 5min before scaling down policies:
- type: Percent value: 10 periodSeconds: 60 # Remove at most 10% of replicas per minute
scaleUp: stabilizationWindowSeconds: 0 # Scale up immediately policies: - type: Percent value:
100 periodSeconds: 15 # Can double every 15 seconds
```

KEDA -- Event-Driven Autoscaling for AI Workloads

KEDA (Kubernetes Event-Driven Autoscaling) is essential for AI inference serving: scale to zero when no requests, scale up based on queue depth or request rate.

```
# Scale LLM inference deployment based on request queue depth: apiVersion: keda.sh/v1alpha1
kind: ScaledObject metadata: name: llm-inference-scaler namespace: ai-serving spec:
scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: llm-inference minReplicaCount: 0 #
Scale to zero when idle maxReplicaCount: 10 cooldownPeriod: 300 # Wait 5min before scaling to
zero triggers: - type: prometheus metadata: serverAddress: http://prometheus:9090 metricName:
inference_queue_depth query: sum(inference_requests_pending) threshold: '5' # Scale up when >
5 queued requests per replica - type: rabbitmq metadata: protocol: amqp queueName:
inference-requests mode: QueueLength value: '10'
```

CHAPTER 13

Scheduling: PriorityClass, Affinity, TopologySpread

PriorityClass -- Workload Preemption

PriorityClasses define the scheduling priority of Pods. Higher-priority Pods can preempt (evict) lower-priority Pods when cluster resources are scarce. This is critical for AI platforms where inference serving (revenue-generating) must preempt batch training jobs.

```
# Define priority tiers: apiVersion: scheduling.k8s.io/v1 kind: PriorityClass metadata: name:
critical-production value: 1000000 globalDefault: false preemptionPolicy: PreemptLowerPriority
description: 'Customer-facing production services' --- apiVersion: scheduling.k8s.io/v1 kind:
PriorityClass metadata: name: ai-inference-serving value: 900000 description: 'LLM inference
endpoints' --- apiVersion: scheduling.k8s.io/v1 kind: PriorityClass metadata: name:
ai-training-batch value: 100000 preemptionPolicy: Never # Batch training: never preempts
others description: 'ML training jobs -- preemptable' # Assign to Pod: spec:
priorityClassName: ai-inference-serving
```

TopologySpread Constraints -- Zone-Aware Spreading

TopologySpreadConstraints distribute Pods evenly across topology domains (availability zones, nodes) for high availability:

```
spec: topologySpreadConstraints: # Spread evenly across availability zones: - maxSkew: 1
topologyKey: topology.kubernetes.io/zone whenUnsatisfiable: DoNotSchedule labelSelector:
matchLabels: { app: api-server } # Also spread evenly across nodes: - maxSkew: 1 topologyKey:
kubernetes.io/hostname whenUnsatisfiable: ScheduleAnyway # Best-effort for node spread
labelSelector: matchLabels: { app: api-server } # maxSkew: max difference in Pod count between
any two topology domains # whenUnsatisfiable: DoNotSchedule = hard requirement #
whenUnsatisfiable: ScheduleAnyway = soft preference
```

CHAPTER 14

Reliability: PodDisruptionBudgets

PodDisruptionBudgets (PDBs) protect applications from being disrupted by voluntary disruptions: node drain, cluster upgrades, Cluster Autoscaler scale-down. They guarantee a minimum number of Pods remain available during disruptions.

```
apiVersion: policy/v1 kind: PodDisruptionBudget metadata: name: api-server-pdb namespace:
production spec: # Option A: minimum available: minAvailable: 2 # At least 2 Pods always
available during disruption # Option B: maximum unavailable: # maxUnavailable: 1 # At most 1
Pod unavailable at any time # Option C: percentage: # minAvailable: 75% # At least 75% of Pods
available selector: matchLabels: { app: api-server } # PDB interaction with node drain: #
kubectl drain node-01 --ignore-daemonsets # Drain honours PDB: evicts Pods only when PDB
allows # If PDB blocks eviction: drain waits (or fails with timeout) # Critical: PDB only
protects against VOLUNTARY disruptions # Node failure (hardware) bypasses PDB # Best practice:
always create PDB for production Deployments # with replicas > 1. Without PDB, all Pods can be
evicted simultaneously.
```

PDB Anti-Patterns

- * **minAvailable: 100% or maxUnavailable: 0:** If PDB requires ALL Pods available, node drains and upgrades block permanently.
- * **PDB with single-replica Deployment:** minAvailable: 1 + replicas: 1 = cluster upgrades blocked forever.
- * **No PDB for stateful services:** Without PDB, all PostgreSQL replicas can be evicted simultaneously during upgrade, causing data unavailability.

CHAPTER 15

Runtime and Policy: RuntimeClass, PodSecurity

RuntimeClass

```
apiVersion: node.k8s.io/v1 kind: RuntimeClass metadata: name: kata-qemu handler: kata-qemu
overhead: podFixed: memory: 120Mi # VM overhead added to all resource calculations cpu: 250m
scheduling: nodeClassification: tolerations: - key: kata-containers operator: Exists effect:
NoSchedule # Use in Pod: spec: runtimeClassName: kata-qemu
```

Pod Security Standards

Policy	Restrictions	Use Case
Privileged	No restrictions	Trusted system workloads: CNI, CSI, GPU operator
Baseline	Block known privilege escalations; restrict hostNetwork/hostPID	General workloads
Restricted	Strict: requires non-root, no privilege escalation, seccomp, dropped caps	Security-critical; PCI, HIPAA

```
# Enforce Pod Security Standards at namespace level: apiVersion: v1 kind: Namespace metadata:
name: production labels: pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/enforce-version: v1.30 pod-security.kubernetes.io/warn: restricted
pod-security.kubernetes.io/audit: restricted
```

CHAPTER 16

Resource Anti-Patterns Reference

Anti-Pattern	Problem	Solution
No resource requests or limits	Pods consume unbounded resources causing noisy-neighbour effects and unpredictable OOM kills on nodes	Set requests and limits on all containers; use LimitRange for defaults; use VPA recommendations
Mutable image tags in production	Deploying :latest or :v1.2 allows image substitution without detection	Pin images to SHA-256 digest; use Kyverno mutateDigest
Running as root	UID 0 in container = host root if container escapes	runAsNonRoot: true; runAsUser: 1000; enforce via PodSecurity Restricted
Privileged containers in production	Full host access equivalent to root on node	Identify minimal capabilities needed; drop ALL; add specific caps
No readiness probe	Service routes to Pod before it is ready to serve	Always implement readinessProbe; health endpoint must check dependencies
StatefulSet with no PodDisruptionBudget	All database replicas can be evicted simultaneously during node drain	PDB minAvailable >= quorum for every StatefulSet
Flat namespace (everything in default)	No isolation, quota, or governance	Namespace per team/environment; RBAC per namespace; ResourceQuota per namespace
ClusterRoleBinding cluster-admin for apps	Application SA has full cluster admin access	Minimum RBAC: only verbs and resources required for the application
No NetworkPolicy (open flat network)	Any Pod can talk to any Pod; lateral movement trivial	Default-deny per namespace; explicit allow rules
Jobs without ttlSecondsAfterFinished	Completed Jobs accumulate; etcd grows unboundedly	Set ttlSecondsAfterFinished on all Jobs; configure CronJob history limits

CHAPTER 17

Hands-On Exercises

Exercise 5.1 -- StatefulSet DNS

Deploy a StatefulSet and verify per-Pod stable DNS:

```
kubectl apply -f statefulset-nginx.yaml kubectl get pods -l app=nginx-sts -w # Verify stable
DNS: kubectl run dns-test --image=busybox:1.36 --restart=Never -it -- sh nslookup
nginx-0.nginx-headless.default.svc.cluster.local # Delete Pod; verify it returns with same
name: kubectl delete pod nginx-1 kubectl get pods -l app=nginx-sts -w
```

Exercise 5.2 -- HPA Load Test

Deploy an HPA and observe CPU-triggered scaling:

```
kubectl create deployment php-apache --image=registry.k8s.io/hpa-example kubectl expose
deployment php-apache --port=80 kubectl autoscale deployment php-apache --cpu-percent=50
--min=1 --max=10 # Generate load: kubectl run load-gen --image=busybox:1.36 --restart=Never
-it -- sh -c 'while true; do wget -q -O- http://php-apache; done' # Watch scale-up in separate
terminal: kubectl get hpa php-apache -w kubectl get pods -l app=php-apache -w
```

Exercise 5.3 -- RBAC Audit

Audit and tighten RBAC permissions for a namespace:

```
# List all bindings: kubectl get rolebindings,clusterrolebindings -A # Check default SA
permissions: kubectl auth can-i --list --as=system:serviceaccount:default:default # Find
cluster-admin bindings (over-privileged): kubectl get clusterrolebindings -o json | python3 -c
" import json,sys d=json.load(sys.stdin) for item in d['items']: if
item['roleRef']['name']=='cluster-admin': for s in item.get('subjects',[]):
print(s.get('name','?'))" # Test NetworkPolicy enforcement: kubectl apply -f
default-deny-all.yaml kubectl run test-client --image=busybox --restart=Never -it -- wget -T 3
http://api-backend
```

End of Part V -- Continue to Part VI: Kubernetes Networking

Part VI delivers the complete networking deep-dive: Pod networking fundamentals, CNI architecture, Calico vs Cilium vs Flannel, eBPF data plane, VXLAN overlays, Service Mesh with Istio and Linkerd, Ambient Mesh, end-to-end packet flow tracing, Gateway API advanced patterns, east-west and north-south traffic, and multi-cluster networking with Submariner and Cilium Cluster Mesh.